

Supervisor Q&A summary: NHS theatre scheduling modelling

This document summarises the main questions we asked during the project, the method we used to answer them, where the evidence is shown, and the result.

Project branch: `improve-operation-length-r2`

Main notebook for modelling results: `notebooks/nbt_operation_length_regression.ipynb`

1. What were we trying to predict?

We worked with three prediction targets.

Target	Type	Meaning
--------	------	---------

---	---	---
-----	-----	-----

<code>meaningful_overrun_flag</code>	Classification	Will the operation meaningfully overrun?
--------------------------------------	----------------	--

<code>operation_length_mins</code>	Regression	How many minutes will the operation take?
------------------------------------	------------	---

<code>duration_error_mins</code>	Regression	How many minutes longer or shorter than planned will the operation be?
----------------------------------	------------	--

The main supervisor discussion focused on `operation_length_mins`.

2. What rule defined overrun and underrun?

First we calculate the difference between actual and planned duration:

```
duration_error_mins = operation_length_mins - ExpectedDurationMins
```

Then we define the tolerance:

```
duration_tolerance_mins = max(0.10 × ExpectedDurationMins, 10 minutes)
```

Then:

```
If duration_error_mins > duration_tolerance_mins ? overrun  
If duration_error_mins < -duration_tolerance_mins ? underrun  
Otherwise ? within_tolerance
```

Simple meaning: we only call something an overrun or underrun if the difference is bigger than the allowed tolerance.

Evidence location: `src/nbt_pipeline/preprocessing/features.py` and statistical notebook target-definition section.

3. What cleaning did we do?

We created one cleaned analysis dataset:

```
result/nbt_smallset_analysis.xlsx
```

Final shape:

```
14,911 rows  
22 columns
```

Main cleaning actions:

? Removed 7 exact duplicate rows.

? Removed free-text fields such as theatre notes.

? Removed staff/consultant identifiers from the primary model because of governance, privacy, high-cardinality and overfitting concerns.

? Removed raw timestamps and reconstructed stage-duration columns from the primary predictors to avoid leakage and unvalidated timing assumptions.

? Kept `operation_start_hour` as a provisional reconstructed feature. It was tested separately and selected for the best operation-length pipeline, but it must still be reported as assumption-sensitive.

? Removed duplicate/superseded outcome columns.

Evidence location: `notebooks/nbt_smallset_data_cleaning.ipynb`.

4. How did we handle missing values?

Technique name:

`Missing-aware preprocessing pipeline`

For categorical variables:

`Missing ? "Missing/not recorded"`

For numeric variables:

`Median imputation + missingness indicator`

For example, if age is missing, the pipeline fills it using the training-fold median and also adds a flag saying age was originally missing.

Important exception:

`ExpectedDurationMins` was required and not imputed.

Why? Because it is the hospital planned duration. If it is missing, replacing it with a median would not have a real scheduling meaning.

Evidence location: modelling notebooks, Section 1 and Section 5.

5. Did we normalise/scale the data?

Yes. Numeric predictors were scaled using:

`StandardScaler`

Formula:

`scaled value = (value - mean) / standard deviation`

This was done inside the model pipeline, not manually on the whole dataset.

Correct order:

1. Split data
2. Fit imputation/encoding/scaling only on training folds
3. Validate model
4. Test once on untouched test set

Why? This avoids data leakage.

Evidence location: modelling notebooks, Section 1.

6. How did we encode categorical variables?

Categorical variables were encoded using:

`OneHotEncoder(handle_unknown="ignore", min_frequency=10)`

This means:

? Categories are converted into model-readable columns.

? Unknown categories in validation/test do not crash the model.

? Very rare categories are handled more safely.

Evidence location: `src/nbt_pipeline/modeling/experiments.py`.

7. Why did we use GroupKFold?

We used GroupKFold because normal KFold can make the model look too good if very similar operation profiles appear in both training and validation.

Simple explanation:

Normal KFold splits rows.
GroupKFold splits groups.

If a group appears in validation, the same group cannot appear in training.

This gives a stricter and more honest validation score.

We tested three fold numbers:

GroupKFold value | CV MAE | CV R² | Test R²

---: | ---: | ---: | ---:

3 folds | 29.93 | 0.701 | 0.732

5 folds | 29.61 | 0.703 | 0.732

10 folds | 29.47 | 0.707 | 0.732

Conclusion: validation was stable across fold choices. The test score did not depend on one arbitrary fold number.

Evidence location: `result/modeling/operation_length_mins/r2_experiments/kfold_number_experiments.csv` and operation-length notebook Section 12.2.

8. Which models did we benchmark?

For `operation_length_mins`, we compared:

? Dummy median benchmark

? Linear regression

? Decision tree

? Random forest

? SVR

? XGBoost

? Neural network

? CatBoost as an extra later check

Main benchmark results:

Model | CV MAE | CV R²

--- | ---: | ---:

XGBoost | 30.75 | 0.683

Neural network | 30.94 | 0.670

Random forest | 30.99 | 0.666

Linear regression | 32.12 | 0.657

Decision tree | 33.60 | 0.618

SVR | 31.22 | 0.617

Dummy median benchmark | 58.07 | -0.089

Conclusion: XGBoost was the best main benchmark model.

Evidence location: operation-length notebook Section 7 and
result/modeling/operation_length_mins/algorithm_comparison.csv.

9. What were the operational benchmarks?

We compared the model against simple baselines.

Benchmark | Test MAE | Test R²

--- | ---: | ---:

Dummy median | 59.98 | -0.106

Hospital ExpectedDurationMins alone | 43.78 | 0.476

Conclusion: the model was much better than the dummy benchmark and better than using the hospital planned duration alone.

Evidence location: operation-length notebook Section 10 and
result/modeling/operation_length_mins/operational_benchmarks.csv.

10. Should we remove ExpectedDurationMins?

We tested this because the supervisor suggested it might improve performance.

Result:

Experiment | Validation R² | Test R² | Test MAE

--- | ---: | ---: | ---:

With ExpectedDurationMins | 0.676 | 0.716 | 30.89 min

Without ExpectedDurationMins | 0.584 | 0.646 | 33.65 min

Conclusion:

Removing ExpectedDurationMins made the model worse.

Reason: ExpectedDurationMins is the hospital's planned estimate of operation length. It is available before the operation and gives the model a strong starting point.

Evidence location: operation-length notebook Section 12.1.

Clickable plots in notebook:

? [Click here to open the validation/test plot.](#)

? [Click here to open the learning / early-stopping plot.](#)

11. Did we use early stopping?

Yes. In the improved experiments, we used early stopping with XGBoost.

Simple meaning:

The model keeps learning while validation error improves.
When validation error stops improving, training stops.

Why? To reduce overfitting.

Evidence location: operation-length notebook Section 12.1 and early_stopping_validation_curve.png.

12. Did we treat skewed operation times?

Yes. We tested:

```
raw_operation_length_mins  
log1p(operation_length_mins)
```

The log-transformed target helped MAE in the early-stopping experiment.

Best supplementary result:

```
XGBoost + log target + early stopping  
Test R2 ~ 0.725  
Test MAE ~ 28.93 minutes
```

Conclusion: log target was useful as a supplementary improvement check, but it did not push R² above 0.78.

Evidence location: operation-length notebook Section 12.2 and `catboost_operation_length_experiments.csv`.

13. Did starting time help?

Yes, slightly.

Trimmed model result:

Experiment | Test R² | Test MAE

--- | ---: | ---:

With operation_start_hour | 0.721 | 29.03 min

Without operation_start_hour | 0.712 | 29.09 min

Conclusion:

```
operation_start_hour improves R2 slightly.
```

But it remains provisional because the reconstructed timestamp has not been independently validated.

Evidence location: operation-length notebook Section 12.2 and `trimmed_columns_start_hour_impact.csv`.

14. Did consultant improve the model?

We tested:

? session_consultant

? listing_cons_code

? both together

Result:

Experiment | Test R² | MAE

--- | ---: | ---:

Best setup without consultant | 0.7319 | 29.63 min

Best + session_consultant | 0.7323 | 29.58 min

Best + listing_cons_code | 0.7313 | 29.53 min

Best + both consultant fields | 0.7246 | 29.85 min

Conclusion: consultant did not materially improve performance and should not be used in the final primary model.

Reason: consultant fields are high-cardinality and governance-sensitive.

Evidence location: operation-length notebook Section 12.2 and `consultant_sensitivity_experiments.csv`.

15. Did full surgery code improve the model?

We tested `actual_proc_1_procedure_code`.

Result:

Experiment | Test R² | MAE

--- | ---: | ---:

Best setup: group + category | 0.7319 | 29.63 min

Best + full procedure code | 0.7326 | 29.45 min

Only full code, no group/category | 0.7158 | 30.31 min

Conclusion: full surgery code gave only a tiny improvement when added, and performed worse when replacing grouped procedure variables.

Reason: the full code has many rare categories.

Audit:

1030 unique full procedure codes

Evidence location: operation-length notebook Section 12.2 and `full_procedure_code_sensitivity_experiments.csv`.

16. What is `actual_proc_1_procedure_code_min20`?

It is a safer version of the full surgery code.

Rule:

If a full procedure code appears at least 20 times, keep it.
If it appears fewer than 20 times, replace it with "Rare procedure code".

This reduces overfitting from very rare codes.

We later tested trimming it.

Experiment | Test R² | Test MAE

--- | ---: | ---:

With `actual_proc_1_procedure_code_min20` | 0.725 | 28.93 min

Without it | 0.721 | 29.03 min

Conclusion: removing it causes only a very small drop, so it can be removed for a simpler model.

Evidence location: operation-length notebook Section 12.2 and `trim_full_code_min20_experiment.csv`.

17. Did procedure chapter help?

Procedure chapter means the first letter of the procedure code.

Example:

S065 ? S

Result:

Procedure representation | Test R² | MAE

--- | ---: | ---:

group + category | 0.732 | 29.51 min

category only | 0.729 | 30.00 min

group only | 0.721 | 29.96 min

chapter only | 0.690 | 31.23 min

Conclusion: chapter only was too broad and performed worse.

Evidence location: operation-length notebook Section 12.2 and procedure_chapter_sensitivity_experiments.csv.

18. Did neural network improve performance?

We tested neural networks with two and three hidden layers.

Neural network | Test R² | Test MAE

--- | ---: | ---:

2 hidden layers: 128-64 | 0.712 | 30.76 min

3 hidden layers: 128-64-32 | 0.716 | 30.46 min

Conclusion: neural network did not beat XGBoost.

Reason: this is tabular hospital data with many categorical variables, and tree-boosting models usually work better for this type of dataset.

Evidence location: operation-length notebook Section 12.2 and neural_network_no_priority_best_columns.csv.

19. Did CatBoost improve performance?

We tested CatBoost because it is strong with categorical/tabular data.

Result:

Model | Target | Test R² | Test MAE

--- | --- | ---: | ---:

XGBoost early stopping | log target | 0.725 | 28.93 min

CatBoost deeper | raw minutes | 0.723 | 30.36 min

CatBoost default tuned | log target | 0.721 | 28.89 min

XGBoost early stopping | raw minutes | 0.715 | 31.12 min

Conclusion: CatBoost was competitive but did not beat the best XGBoost setup.

Evidence location: operation-length notebook Section 12.2 and catboost_operation_length_experiments.csv.

20. Which features were most important?

We used permutation importance.

Simple meaning:

Shuffle one column and see how much worse the model gets.

Top features:

Rank | Feature | MAE gets worse by

---: | --- | ---:

1 | ExpectedDurationMins | +25.84 min

2 | anaesthetic_desc | +4.11 min

3 | procedure_code_group | +3.57 min

4 | intended_management_label | +3.53 min

5 | procedure_code_category | +2.16 min

Conclusion: planned duration, anaesthetic type and procedure type were the strongest predictors.

Evidence location: operation-length notebook Section 12.2 and latest_best_feature_importance.csv.

21. What are the best columns for operation_length_mins?

Recommended clean practical column set:

```
ExpectedDurationMins
sex_national_code
age_at_operation
ASAScore
anaesthetic_desc
admission_type_label
intended_management_label
procedure_code_group
procedure_code_category
operation_start_hour
TheatreRoom
session_specialty
```

Optional improvement/sensitivity column:

```
actual_proc_1_procedure_code_min20
```

Not recommended for final primary model:

```
session_consultant
listing_cons_code
raw actual_proc_1_procedure_code
procedure_code_chapter only
operation end time
duration_error_mins
duration_status
meaningful_overnrun_flag
stage duration columns
```

22. What was the best score?

Final selected operation-length pipeline result:

```
operation_length_mins
XGBoost
Test MAE ~ 30.45 minutes
Test R2 ~ 0.712
```

The conservative official notebook benchmark before selecting the strongest setup was around:

```
R2 ~ 0.72-0.73
MAE ~ 29 minutes
```

Highest earlier R² sensitivity result:

```
R2 ~ 0.732
```

Best early-stopping/log-target result:

```
R2 ~ 0.725
MAE ~ 28.93 minutes
```

23. Why did we not reach R² > 0.78?

We tried many reasonable model and feature options, but performance plateaued around 0.72–0.73.

This suggests the limit is probably missing information, not only the algorithm.

Useful missing data may include:

? case complexity indicators;
? all secondary procedure codes;
? case order within theatre list;
? staffed session start/end time;
? cancellation and delay reasons;
? equipment or bed delays;
? validated theatre-flow timestamps;
? governed team/session-level information.

Supervisor explanation:

The model is useful but not perfect. To reach higher accuracy, the dataset likely needs richer operational and case-complexity v

24. Final presentation message

The project followed a strong modelling process:

1. Cleaned the data and removed leakage.
2. Defined clear targets.
3. Used missing-aware preprocessing.
4. Encoded and scaled data inside training folds.
5. Used grouped validation to avoid overly optimistic scores.
6. Compared multiple models against benchmarks.
7. Tested supervisor questions with evidence.
8. Added plots for validation/test comparison and early stopping.
9. Explained why the model score plateaued.

Final statement:

XGBoost was the strongest overall model for operation length. ExpectedDurationMins, anaesthetic type and procedure type were the

25. Sources and references

Project data source

Source | How it was used in this project

--- | ---

Internal NBT theatre scheduling dataset supplied for the project | Main source for operation records, expected duration, recorded operation length, procedure information, admission/management labels, anaesthetic type, theatre room and derived modelling targets. The original source file was not overwritten.

Project notebooks and result files in this repository | Source for all reported figures, tables, model scores and sensitivity checks.

Method and software references

Topic used in the project | Reference

--- | ---

pandas data cleaning and tabular processing | The pandas development team. pandas documentation. <https://pandas.pydata.org/docs/>

NumPy numerical operations | Harris, C. R. et al. (2020). Array programming with NumPy. Nature. <https://numpy.org/doc/stable/>

scikit-learn pipelines, preprocessing, metrics and model comparison | Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research. <https://scikit-learn.org/stable/>

GroupKFold grouped cross-validation | scikit-learn documentation: GroupKFold. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupKFold.html

Missing-value imputation inside the training pipeline | scikit-learn documentation: SimpleImputer.
<https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html>

One-hot encoding of categorical variables | scikit-learn documentation: OneHotEncoder.
<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>

Standardisation / scaling of numerical variables | scikit-learn documentation: StandardScaler.
<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html>

Permutation feature importance | scikit-learn documentation: permutation_importance.
https://scikit-learn.org/stable/modules/generated/sklearn.inspection.permutation_importance.html

XGBoost model | Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. <https://arxiv.org/abs/1603.02754>

CatBoost sensitivity model | Prokhorenkova, L. et al. (2018). CatBoost: unbiased boosting with categorical features.
<https://arxiv.org/abs/1706.09516>

Neural network benchmark | scikit-learn documentation: MLPRegressor and MLPClassifier.
https://scikit-learn.org/stable/modules/neural_networks_supervised.html

Statistical modelling and logistic regression support | statsmodels documentation. <https://www.statsmodels.org/stable/index.html>

Plotting | Matplotlib documentation and Seaborn documentation. <https://matplotlib.org/stable/> and <https://seaborn.pydata.org/>

Important citation note

The overrun/underrun tolerance rule used here is a project working definition, not an official NBT policy threshold. The rule was:

```
duration_error_mins = operation_length_mins - ExpectedDurationMins  
  
duration_tolerance_mins = max(10 minutes, 10% of ExpectedDurationMins)  
  
meaningful_overrun_flag = 1 when duration_error_mins > duration_tolerance_mins
```

This definition should be confirmed with the operational team before it is used outside the project.